Ders 4 Genişletilmiş Notu: Facade, Flyweight, Proxy, Chain of Responsibility ve Command Kalıpları

Ders 4 Genişletilmiş Notu: Facade, Flyweight, Proxy, Chain of Responsibility ve Command Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Facade Kalıbı
- 2. Flyweight Kalıbı
- 3. Flyweight Örneği: Kimya Fabrikası
- 4. Proxy Kalıbı
- 5. Proxy Örneği: İzle-Öde Yayın Sistemi
- 6. Davranışsal Kalıplara Geçiş ve Chain of Responsibility
- 7. Chain of Responsibility Örneği: Reklam Yönetim Yazılımı
- 8. Command Kalıbı
- 9. Command Örneği: Akıllı Ev Kumandası
- 10. Örnek Sorular

1. Facade Kalıbı

Geçen hafta Decorator kalıbında kalınmıştı. Ders, yapısal (structural) kalıplardan Facade ile devam etti. Hoca kalıbın adının okunuşunu da açıkladı: kelime “fasad” diye okunur ve binalara yapılan ön yüz kaplamasından (cephe) gelir; “face” ile akrabadır ve bir şeyin ön yüzünü oluşturmak anlamına gelir. Kalıp, adına çok uygun düşer.

Slayttaki amaç tanımı şöyledir (ekrandan, 00:07:40): çeşitli alt sistemlerde bulunan farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamak. Böylece istekçiler, alt sistemlerle ilgili alt düzey ayrıntılardan soyutlanabilir.

Kalıbın çıkış noktası, iyi tasarlanmamış ve bu yüzden biraz karmaşık bir alt sistemdir. Sistemde yapılması gereken işler çeşitli sınıfların çeşitli metotları üzerinden yürür ve bu metotlar birbirinden çok farklı sınıflara dağılmıştır. İstekçileri “şu sınıfın şu metodu, bu sınıfın bu metodu” diye uğraştırmak yerine, bütün sisteme ortak bir ön yüz sunan bir sınıf yazılır. Bu sınıfta sistemin en temel işlevlerini başlatan metotlar bulunur; bu metotlar da alt sistemdeki ilgili sınıfların metotlarını çağırır. Bu sınıfa Facade denir.

Slayttaki kalıp yapısında (ekrandan, 00:07:40) üstte birkaç istekçi sınıf (client classes), ortada tek bir `Facade` kutusu ve altta “subsystem classes” çerçevesi içinde kalıtım ve ilişki çizgileriyle bağlı birçok sınıf vardır. İstekçilerin hepsi yalnız `Facade`’a bağlanır; `Facade` ise alt sistemdeki birkaç sınıfa oklarla bağlıdır.

Hoca kalıbı “bu kadar, başka bir şey yok” diye özetledi. Facade biraz MVC’ye, biraz Adapter’a benzeyen bir fikir düzeyinde kalır. Bu yüzden ayrıca bir örnek yapılmadı.

2. Flyweight Kalıbı

2.1 Amaç ve metin editörü örneği

Flyweight, boks sıkletlerinden gelen bir addır; Türkçede “tüy sıklet” ya da “sinek sıklet” diye çevrilebilir. Kalıbın amacı (ekrandan, 00:10:00): paylaşım yolu ile çok sayıda basit nesnenin kullanımının desteklenmesi.

Örnekte bir metin düzenleme programı yazılmaktadır. Programın, font ve diğer biçimlendirme bilgilerine göre metni çok net göstermesi (excellent rendering) istenir. En iyi sonucu almak için her harf bir grafik nesnesi olarak ele alınır: grafik nesnesi ne kadar ayrıntılı çizilirse hedefe o kadar yaklaşılır. Her harf yüksek çözünürlüklü ve yüksek bellek kaplayan bir nesne olur.

İşi basitleştirmek için sabit genişlikli bir font, yani bir daktilo düşünülebilir: kağıt boyutuna göre belli sayıda satır ve sütun vardır. Font değişken genişlikli olsa bile bir satıra kaç harf sığacağı hesaplandıktan sonra “3. satırdaki 5. karakter” denebilir. Bu “3. satırdaki 5. sütun” ile aynı şey değildir, çünkü harflerin genişliği aynı olmayabilir; ama konum yine bu biçimde saklanabilir. Metin, sütunlar ve satırlar içinde bulunan harfler olarak ele alınabilir.

Sorun şudur (ekrandan, 00:10:00): her harf ayrı bir nesne örneği olarak ele alınırsa küçük bir belgede bile binlerce nesne olur ve bu nesnelerin bellekte ve ikincil saklama ortamlarında kaplayacağı yer çok aşırı olur. Nesneler dosyaya birebir yazılırsa dosya boyu çok artar. Dosyada nesne değil de yalnız harf adı saklansa, örneğin HTML ile CSS gibi bir biçim kullanılsa bile, belge belleğe yüklenince her harfin resmi de ister istemez yüklenir.

Hocanın sayısal örneği: her ders açılırken hazırlanan üç sayfalık ders öneri formunda boşluklarla birlikte 5000 kadar karakter vardır. Boşluklar yüzde 10-15 kadar tutsa geriye yaklaşık 4000 harf kalır. Bu sayı, her harfin kaç megabayt tuttuğuyla çarpılırsa çok büyük bir rakam çıkar.

Çözüm: Unicode kümesindeki her harf bir harf nesnesi olarak ele alınır; harfin bir adı ve bir bitmap’i olur. Metin içinde harfin geçtiği her yere bu nesne gömülmez. Bunun yerine bellekte bir harf havuzu tutulur: harflerin, noktalama işaretlerinin ve diğer simgelerin her birinden yalnız bir nesne. Metin içinde ise belirli bir satırdaki belirli bir eleman için yalnız bu nesneye bir işaretçi tutulur. İşaretçi 4 bayt, bilemediniz 8 bayt tutar. Böylece bir harf için megabaytlarca bilgi yerine yalnız bir işaretçi saklanır ve bellekten epey tasarruf edilir. C’de olduğu gibi işaretçi kullanılabilir ya da havuzdaki nesnelere bir map ile erişilir: anahtar harfin adıdır, “A” anahtarı A nesnesini getirir.

Paylaşılan harf nesnelerine flyweight denir. Hoca burada bir inceliğe dikkat çekti: aslında hafif olan nesnelerin kendisi değildir, nesneler hacimlidir. Hafif olan, o nesnelere ulaşmak için kullanılan referanslardır (işaretçilerdir).

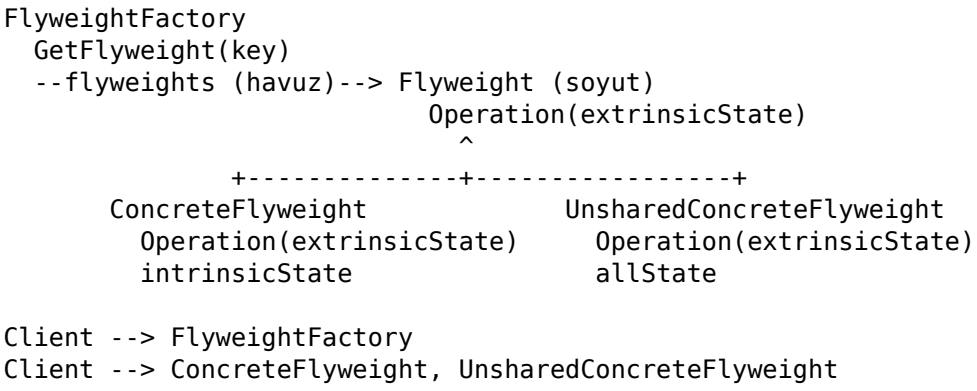
2.2 Metin yapısı

Çözümün tasarımında (ekrandan, 00:15:30) soyut bir `Glyph` sınıfı vardır: `Draw(Context)` ve `Intersects(Point, Context)`. Altında üç sınıf yer alır: `Row`, `Character` ve `Column`. Üçü de bu iki metodu gerçekler; `Character` ayrıca `char c` alanını tutar. `Row` ve `Column`, `children` adlı bir topluluk ile `Glyph` nesneleri içerir.

Belge çok sayıda satırdan oluştuğu ve “belirli bir satırdaki belirli bir karakter” diyebilmek istendiği için burada Composite’i andıran bir yapı vardır. Satırlar sütun, sütunlar satır ve ikisi de karakter içerebilir: kökte satırlar, ara düğümlerde sütunlar ve yapraklarda karakterler olabilir ya da tam tersi; yapı iki düzene de izin verir. Bir satır çizileceği zaman içindeki bütün harfler çizilir. Composite’ten farkı, parça-bütün ilişkisinde nesnelerin kendisinin değil, yalnız flyweight işaretçilerinin yer almasıdır. `Character`, kendi karakter kodunu anahtar olarak kullanıp havuzdan çizimi yapacak asıl nesneyi alır. Slaytta havuz gösterilmemiştir, ama bir yerde tutulmak zorundadır.

2.3 Kalıp yapısı

Genelleştirilmiş kalıp yapısı şöyledir (ekrandan, 00:20:00):



`GetFlyweight(key)` için not kutusundaki sözde kod:

```
if (flyweight[key] exists) {
    return existing flyweight;
} else {
    create new flyweight;
    add it to pool of flyweights;
    return the new flyweight;
}
```

Şemada açıkça bir fabrika (FlyweightFactory) vardır. Fabrika metodu verilen anahtara karşılık gelen flyweight'i döndürür. Slaytta dönüş türü gösterilmemiş, ama koddan bunun `Flyweight` türünde olduğu anlaşılır. Ortak üst tür soyut çizilmiştir; içine hiçbir somut metot ya da alan konmayacaksa bir arayüz (interface) olarak da yazılabilir. Yalnız anahtardan yola çıkarak ilgili flyweight oluşturulabiliyorsa, anahtar havuzda yoksa `else` kolunda oluşturulur. Böyle bir oluşturma mümkün değilse `GetFlyweight` metodu null döndürecek biçimde kodlanır.

Hoca, şemada istemciler ile fabrika arasında da bir ok çizilmesinin yararlı olacağını söyledi. Nesneler açısından bakıldığında fabrika flyweight'leri içerir; istemci bir flyweight'e erişmek istediğinde onu fabrikadan ister ve işaretçisini öyle alır.

Paylaşılması gerekmeyen bazı flyweight'ler varsa, havuzda yer almayacak bir alt sınıf (`UnsharedConcreteFlyweight`) oluşturulabilir. Ama genelde ne kadar çok paylaşım yapılırsa o kadar çok tasarruf sağlanır.

2.4 İçsel ve dışsal durum

Kalıbın tasarruf sağlamasının en önemli koşulu, flyweight'in durum bilgisinin içsel (intrinsic) ve dışsal (extrinsic) olarak ikiye ayrılabilmesidir. Metin örneğinde içsel durum, harfin büyük kapasiteli çizim verisidir. Dışsal durum ise harfin hangi satır ve sütuna yazılacağı bilgisidir. Bu ikiye ayırma yapılamıyorsa kalıp tasarruf sağlamaz.

Dışsal durum, bir harfin metinde geçtiği her yer için ayrı ayrı tutulur ve bu yüzden çok sayıda tekrarlanır. Ancak iki tam sayıdan ibaret küçük bir bilgi olduğu için, harfin resmine oranla çok küçüktür; çok sayıda tekrarlanması yine kazanç sağlar.

Satır ve sütun bilgisi hesaplanabilir bir bilgidir; bu hesaplanabilir bilgiden yola çıkarak sabit bilgi, yani yüksek çözünürlüklü bitmap grafiği eşleştirilir. Bazı kaynaklarda intrinsic yerine immutable (değişmez), extrinsic yerine mutable (değişebilir) denir. Belli bir fonttaki A harfinin resmi bir kez çizildi mi değişmez: bir fontta A harfi şapkalı, başka bir fontta yuvarlak gövdeli ve yanında düz çizgili olabilir, ama o fontta hep aynıdır. Farklı boyutlar (ör. 14 ve 18 punto) için büyük olasılıkla ayrı bitmap'ler saklanır. Harflerin hangi satırda ve sütunda yer aldığı ise dışarıdan hesaplanır ve değişebilir. İçsel kısım isterseniz `final` olarak tanımlanabilir; bu gerekli de olabilir, olmayabilir de.

Sağlanan tasarrufun oranı şununla orantılıdır: içsel bilgi ne kadar hacimli, dışsal bilgi ne kadar hafifse tasarruf o kadar büyüktür. İkisinin boyutu birbirine yaklaşıyorsa kalıbı kullanmaya hiç gerek yoktur.

2.5 Flyweight ile Singleton farkı ve iç sınıflar

Flyweight biraz Singleton'a benzer, ama tam olarak değil. Singleton'da bir sınıftan tek nesne vardır. Flyweight'te de A harfinden tek nesne vardır, ama o nesneye işaret eden ve dışsal durum bilgisini taşıyan çok sayıda işaretçi vardır. Durum bilgisinin bir kısmı değişmez, bir kısmı değişir.

> **Hocanın vurgusu:** Flyweight ile Singleton arasındaki bu fark gözden kaçırılmamalıdır.

Flyweight nesnelerine yalnız havuz üzerinden erişilmesi sağlanabilir mi? Evet: flyweight'ler fabrika sınıfının iç sınıfı (inner class), hatta private iç sınıfı olarak tasarlanırsa dışarıdan erişim iyice kısıtlanır. Hoca örnek kodun da bu biçimde yazıldığını söyledi.

2.6 Kullanım koşulları ve nesne kimliği

Kalıbı kullanmak için şu koşulların tümü gerekir:

- Uygulama çok sayıda nesne kullanır.
- Nesne sayısı çok olduğu için saklama masrafları yüksektir (ör. ayrıntılı bitmap'lerin küçük, orta, büyük boy fontlar için saklanması).
- Nesnenin durum bilgisinin önemli bir kısmı dışsallaştırılabilir, bir kısmı da içsel kalabilir. Hoca burada "önemli kısmı" ifadesinin hacim değil nicelik anlamında olduğunu belirtti: A harfi bin yerde geçiyorsa bin konum bilgisi vardır. Hoca slayttaki çevirinin çok iyi olmadığını, "dışsallaştırma" için daha iyi bir terim aranabileceğini de not etti.
- Dışsal durum ayrılınca çok sayıda nesne grubunun yerine az sayıda paylaşılan nesne kullanılabilir (havuz üzerinden paylaşım).
- Nesne kimliği (object identity) önemli değildir.

Son koşul için hoca şu açıklamayı yaptı. Slayttaki bütün küçük "a" harfleri bellekte aynı nesnedir: ilk satırdaki ilk a ile aynı satırdaki ikinci a aynı nesnedir. Bazı durumlarda ise nesnelerin farklı olması istenir. Bir hava yolu şirketi "bu ay bir milyon yolcu taşıdık" diyor olsun. Bu bir milyon tekil yolcu mu, yoksa toplam seyahat sayısı mı? Biri ayda üç kez, biri beş kez, biri bir kez, biri on kez uçmuş olabilir. Ortalamada bir kişi ayda iki kez uçuyorsa, bir milyon bilet satılmıştır ama tekil müşteri sayısı 500.000'dir. Önemli olan tekil müşteriyse, her insanın TC kimlik numarası gibi ayrı bir tanımlayıcısı vardır. Böyle bir problem Flyweight ile modellenirse bu kişilerin ayrı kişiler olduğu bilgisi kaybolur.

Nesne kimliği derken, çalışma ortamının nesneye atadığı ve programcının değiştirmediği, değiştiremediği evrensel tekil tanımlayıcı (universal unique identifier) kastedilir. Flyweight'te nesne kimliği kaybolur. Bu önemliyse Flyweight kullanılamaz, önemli değilse kullanılabilir.

3. Flyweight Örneği: Kimya Fabrikası

3.1 Problem ve ilk çözüm

Bir kimya fabrikası işletilmektedir ve üretim bilgi sistemi hazırlanacaktır. Fabrikada birçok kimyasal madde üretilir. Birkaç kimyasal maddenin birleşiminden çok sayıda karışım da hazırlanır ve aynı madde doğal olarak birden fazla karışımda yer alabilir. Hoca örneği Design Patterns Java Workbook kitabından aldığını, örneğin kitabın kullandığı baskısında 328. sayfada başladığını söyledi.

Problem tanımındaki "madde" ve "bileşik" sözcükleri akla Composite'i getirebilir, ama hoca hemen öyle düşünmemeyi istedi; problem ona göre düzenlenecekti.

İlk akla gelen çözüm iki sınıftan oluşur (ekrandan, 00:30:00):

```
KimyasalMadde
  - gram : double
  - isim : String
  - simge : String
  - atomAgirligi : double
  + Kimyasal(isim : String, simge : String, atomAg : double)
  + setGram(double)
```

```
  + getGram() : double
  + getIsim() : double
  + getAtomAgirligi() : double
  + getMol() : double        // not: return gram/atomAgirligi;

Karışım  (KimyasalMadde'leri içerir)
  - isim : String
  - maddeler : LinkedList<KimyasalMade>
  + setIsim(String)
  + getIsim() : String
  + maddeEkle(KimyasalMadde)
  + tarifEt() : String
```

Kimyasal maddenin bir adı (ör. potasyum nitrat), farklı karışımlarda farklı olacağı için bir gramajı, periyodik tablodaki kimyasal simgesi ve atom ağırlığı vardır. Kimyacılar mol cinsinden hesap yapmak isteyebileceği için atom ağırlığı da gereklidir. Kurucu bütün üye alanları alır; bütün alanlar için getter metotları vardır. Setter yalnız gram içindir, çünkü simge ve atom ağırlığı sabittir, asla değişmez. `getMol()` metodu kimyasal tanımı gereği gramın atom ağırlığına bölümünü hesaplar.

> **Ders sonrası not:** Slayttaki şemada `getIsim()` dönüş türü `double` yazılmıştır (ekrandan, 00:30:00 ve 00:40:00); isim `String` olduğu için doğrusu `+ getIsim() : String` olmalıdır. Aynı slayttaki `KimyasalMade` yazımı da `KimyasalMadde` olmalıdır.

Karışım sınıfının da kendine özgü bir adı ve içerdiği maddeler vardır; maddeler bağlı liste (linked list) olarak tutulur. Ad için getter ve setter, karışıma madde eklemek için `maddeEkle` ve "bu karışımda şu maddeden şu kadar, bu maddeden bu kadar" biçiminde tarif veren `tarifEt` metodu vardır.

Slayttaki örnek: 100 gr barutta 75 gr potasyum nitrat, 15 gr karbon ve 10 gr kükürt bulunur (ekrandan, 00:30:00). Toplam $75 + 15 + 10 = 100$ gr eder.

3.2 Neden Composite değil?

Composite, hiyerarşik bir parça-bütün ilişki kümesini modellemek için tercih edilir. Karışımda böyle bir hiyerarşi yoktur: bir şeyin içine bir şey eklenmez, kabın içine biraz ondan, biraz bundan konur. Ekleme sırası önemliyse bağlı listede eklenme sırası bunu modelleyebilir. Önce bir ara birleşim oluşturulup sonra o birleşimler bir araya getiriliyor olsaydı, belki Composite tercih edilebilirdi. Bu nedenlerle bu problem için Flyweight çok daha uygundur.

3.3 İlk çözümün sorunu

Bu çözüm barut gibi bileşikleri destekler. Ancak karbon başka karışımlarda başka gramajla da bulunacaktır (ekrandan, 00:30:00). Diğer karışımlardaki karbon kopyalarında yalnız gram bilgisi farklı, kalan durum bilgisi tamamen aynı olacaktır. Her karışım için yalnız gram bilgisi farklı olan çok sayıda karbon nesnesi mi oluşturulmalıdır?

Hoca bunun aşırı bir bellek israfına yol açmayabileceğini kabul etti: bir fabrika kaç karışım üretebilir ki? Bin karışım olsa ve madde nesnesi kabaca bir kilobayt tutsa, toplam bir megabayt eder; bu da bugünkü bilgisayarlar için büyük bir kapasite değildir. Onu rahatsız eden asıl şey bakımdı. Diyelim ki karbonun simgesi yazılırken C yerine yanlış bir harf (ör. K; o da zaten potasyumun simgesidir) kullanıldı. Aynı madde artık farklı simgeli nesnelerle temsil edilir; hangisi bulunup düzeltilecek, hangisi düzeltilmeyecek? Bu tür bakım zorlukları ortaya çıkar.

3.4 İkinci çözüm: değişeni sarmalamak

En temel ilke olan "neyin değiştiğini bul ve onu sarmala" ilkesine daha uygun bir çözüm aranır. Gram bilgisi değişir, diğer bütün bilgiler aynı kalır. Değişen gramaj ayrı bir sınıfın üye alanı yapılmalı, geri kalan bilgiler başka bir sınıfta tutulmalıdır (ekrandan, 00:30:00: "Gram bilgisi madde sınıfından ayrılmalı, ayrı bir sınıfta yer almalıdır").

Henüz kalıp kullanmadan, daha doğru bir nesneye dayalı tasarımla karışım ile kimyasal madde arasına bir `Bileşen` sınıfı eklendi. Gram bilgisi bileşene alındı. Bir karışım birden fazla bileşen

içerir, ama her bileşen tek bir kimyasal maddeyle ilişkilidir. Böylece aynı kimyasal madde iki karışımda farklı gramajlarla kullanılıyorsa, iki karışım nesnesi ve aynı maddeyle ilişkili iki ayrı bileşen nesnesi olur; ama hepsinde aynı kimyasal madde nesnesi yer alır. Hoca bunu epey doğru bir tasarım olarak niteledi ve kullanılabileceğini söyledi.

3.5 Flyweight ile çözüm

Değişen gram bilgisinin dışarıdan hesaplanıp verilen dışsal (extrinsic) bir bilgi olduğu fark edilince, problemi bir de Flyweight ile modelleyip ne kazanılıp ne kaybedildiği karşılaştırıldı. Çözümün şeması (ekrandan, 00:40:00):

```
Karışım  --içerir-->  Bileşen  --1:1-->  KimyasalMadde
   |
   +--kullanır-->  MaddeFabrikası

Bileşen
  - gram : double
  - madde : KimyasalMadde
  + Bilesen(double, KimyasalMadde)
  + getGram() : double
  + getMol() : double
  + getMaddeAdi() : String

KimyasalMadde
  - isim : String
  - simge : String
  - atomAgirligi : double
  + Kimyasal(isim : String, simge : String, atomAg : double)
  + getIsim() : double
  + getAtomAgirligi() : double
  + getMol(gram : double) : double

Karışım
  - isim : String
  - maddeler : LinkedList<KimyasalMadde>
  + setIsim(String)
  + getIsim() : String
  + maddeEkle(simge : String, gram : double)
  + tarifEt() : String

MaddeFabrikası
  - maddeler : HashMap<String, KimyasalMadde> {static}
  + maddeBul(simge : String) : KimyasalMadde {static}
```

Önce Flyweight'in fabrikası kuruldu. Değişen şey `maddeEkle` metodudur: önceki çözümde parametre olarak madde nesnesinin kendisini alıyordu; bu çözümde maddenin tekil tanımlayıcısı olan kimyasal simgeyi (string) ve gramı alır (slaytta kırmızı çerçeveyle işaretli). Simge madde fabrikasına verilir, fabrikanın `maddeBul` metodu bir tür kayıt defteri (registry) gibi davranıp ilgili kimyasal maddeyi bulur ve döndürür.

Kolaylık için fabrikanın maddeler tablosu ve `maddeBul` metodu static tanımlandı; doğrudan sınıf üzerinden erişilir. Farklı fabrika nesneleri gerekseydi böyle yapılmazdı. Hocanın gerekçesi: karbon karbondur. Farklı tür karbonlar olsaydı bir fabrika bir türü, öbür fabrika öbür türü veriyor diye düşünülebilirdi; ama karbon her durumda aynı maddedir, bu yüzden tek, static bir fabrika yeterlidir.

Bileşen sınıfında ve kimyasal madde sınıfında başka bir değişiklik yoktur. Böylece kimyasal madde nesneleri bileşen ve karışım üzerinde bir bütün olarak kullanılmak yerine, karışımda yalnız simgeleriyle kullanılmış olur.

Hoca, slayttaki şemada `Karışım` sınıfının `maddeler` alanının hatalı kaldığını fark etti: alan kimyasal madde değil, bileşen listesi olmalıdır; kodda doğrusu `LinkedList<Bilesen>` yazılmıştır.

Düzeltmeyi daha önce işaretlediğini ama yapmayı atladığını söyleyip slaytları düzeltti ve kodda doğrusunun esas alınmasını istedi. Alanın adının da "maddeler" yerine "bileşenler" olmasının daha doğru olacağını not aldı (Eclipse'te Refactor > Rename ile yapılabilir).

3.6 Karşılaştırma

Tasarım açısından bakıldığında Flyweight çözümü ikinci çözüme göre çok fazla bir şey kazandırmadı. Bir avantajı, madde fabrikasının fabrikada sahip olunan maddelerin kataloğunu tutmasıdır. Alt düzey kodlama açısından ilk çözümde madde nesnesi birden fazla yerde saklanıyordu; ama kullanılan dil ya da optimize edici zaten yalnız referans saklıyorsa bu da çok fark ettirmez. Yine de kalıbın bütün parçaları bir örnekte görülmüş oldu. Hocaya göre özgün nesneye dayalı tasarıma göre çok şey değişmese de, kalıplarla düşünüldüğünde bu çözüme doğrudan ve daha kısa sürede ulaşılabilir.

3.7 Kod

Eclipse'teki projede üç çözüm planlanmıştı: yalnız iki sınıflı en kötü çözüm, nesneye dayalı ilkelere uyan çözüm ve kalıp kullanan çözüm. Derste yalnız üçüncüsünün (`dp.flyweight.solution3`) kodu vardı; hoca ilk iki çözümün kodunun yazılmasını not etti ve bunu alıştırma olarak bıraktı.

Madde fabrikası (ekrandan, 00:44:40; satırların sağı ekranda kesik):

```
class MaddeFabrikasi {
    private static HashMap<String, KimyasalMadde> maddeler = new HashMap<...>();
    static {
        maddeler.put("C", new KimyasalMadde("Karbon", "C", 12.0107));
        maddeler.put("S", new KimyasalMadde("Kükürt", "S", 32.066));
        maddeler.put("KNO3", new KimyasalMadde("Potasyum Nitrat", "KN...
        maddeler.put("NaNO3", new KimyasalMadde("Sodyum Nitrat", "NaN...
    }
    public static KimyasalMadde maddeBul(String simge) {
        return maddeler.get(simge);
    }
}
```

Tablonun HashMap'inde anahtar maddenin simgesi, değer maddenin kendisidir. Tablonun üretilen maddelerle doldurulması gerekir. Bu iş normalde kurucuda yapılırdı, ama her şeyin static olması istendi. Kurucunun yerine geçebilecek bir şey Java'da vardır: doğrudan `static` yazıp bir blok açılır ve sınıfın static kodları içine konur. Sınıf belleğe yüklendiğinde ilk anda bu static blok yürütülür. Bu blokta nesne metotları çağrılamaz, çünkü o anda sınıftan hiçbir nesne oluşturulmamıştır.

Karışım sınıfı (ekrandan, 00:46:50; satır sonları ekranda kesik):

```
public class Karisim {
    private String isim;
    private LinkedList<Bilesen> maddeler = new LinkedList<Bilesen>();
    public String getIsim() { return isim; }
    public void setIsim(String isim) { this.isim = isim; }
    public void maddeEkle(String simge, double gram) {
        maddeler.add(new Bilesen(gram, MaddeFabrikasi.maddeBul(simge)));
    }
    public String tarifEt() {
        String tarif = isim;
        for (Bilesen madde : maddeler)
            tarif += "\n" + madde.getGram() + "gr " + madde.getMa...;
        return tarif;
    }
}
```

`maddeEkle`, verilen simgeden yola çıkarak maddeyi fabrikadan alır, gramı da ekleyip bir bileşen oluşturur ve bağlı listeye ekler. `tarifEt`, listedeki her bileşenin gramını ve madde adını bir

metinde biriktirip döndürür. `Bilesen` (gram ve ilişkili kimyasal madde) ile `KimyasalMadde` basit sınıflardır.

Ana program (ekrandan, 00:50:00):

```
public static void main(String[] args) {
    Karisim karaBarut = new Karisim();
    karaBarut.setIsim("Kara barut");
    karaBarut.maddeEkle("NaNO3", 75);
    karaBarut.maddeEkle("C", 15);
    karaBarut.maddeEkle("S", 10);
    System.out.println(karaBarut.tarifEt());
    Karisim temizBarut = new Karisim();
    temizBarut.setIsim("Temiz barut");
    temizBarut.maddeEkle("KNO3", 75);
    temizBarut.maddeEkle("C", 20);
    temizBarut.maddeEkle("S", 25);
    System.out.println(temizBarut.tarifEt());
}
```

İki tür barut üretiliyor: kara barut ve daha temiz yanan “temiz barut”. İkisinde de karbon ve kükürt vardır, ama karbon birinde 15, diğerinde 20 gram; kükürt birinde 10, diğerinde 25 gram kullanılır. Çalıştırılınca çıktı şöyledir (ekrandan, 01:50:00, konsolda kalan çıktı):

```
Kara barut
75.0gr Sodyum Nitrat
15.0gr Karbon
10.0gr Kükürt
Temiz barut
75.0gr Potasyum Nitrat
20.0gr Karbon
25.0gr Kükürt
```

Bu örnekte Flyweight çok fazla tasarruf yapmadı. Ama zamanla üretilen karışımlar çoğalır ve aynı madde bu karışımlarda daha sık yer alırsa tasarruf oranı artar.

> **Hocanın vurgusu:** Flyweight kalıbının kullanılmaya başlanmasını geleceğe yönelik küçük bir altyapı yatırımı gibi düşünün. İyi bir modelleme ile Flyweight kalıbı arasındaki tek fark ek bir fabrika sınıfıdır; bu da fazla kod ya da bakım yükü getirmez, ama tasarruf potansiyeli yüksektir.

4. Proxy Kalıbı

4.1 Amaç ve resim vekili örneği

Aradan sonra Proxy (vekil) kalıbına geçildi. Amaç, bir nesneye erişimi denetlemek için o nesnenin yerine geçecek bir vekil nesneyi araya koymaktır.

Örnek: ekrana resim çizmek zaman alıcı bir işlem olabilir; eski bilgisayarlarda böyleydi. Bir belgeye çok sayıda ve yüksek çözünürlüklü resim eklenmişse, dosya açılıp hızlıca ya da belli bir tempoyla aşağı kaydırılırken küçük duraklamalar olurdu ve bunların resimlerden kaynaklandığı anlaşılırdı. Resmin yerinde önce boş bir dikdörtgen görünür, sonra duruma göre resim gösterilir ya da çok hızlı kaydırılıyorsa gösterilmeden geçilirdi. Bilgisayarların performansı o zamandan beri çok arttı.

Bu davranış Proxy kalıbıyla kodlanıyordu. Kelime editörü doğrudan resmi işaret etmez, bir resim vekilini işaret eder. Resim vekili, resmin hangi dosyada olduğunu (yolunu) tutar. Resim henüz yüklenmemişse vekil gidip resmin fiziksel dosyasından ya da belgenin içinde gömülü olduğu yerden onu okur. İş hızlıysa fark edilmeden geçilir; yavaşsa ya da aynı anda birkaç program açıksa, örneğin bir yerden bir yere içerik kopyalanıyorsa, gecikme hissedilir.

4.2 Ortak arayüz ve örnek çözüm

Proxy kalıbının kullanılabilmesi için vekilin, vekili olacağı nesneyle aynı arayüze, yani aynı metot imzalarına sahip olması gerekir. Örnekte resmin hangi public metotları varsa vekilde de o metotlar olmalıdır; başka türlü vekil olamaz. İstekçi ister vekille ister resimle konuşsun, aynı metotları çağırabilmeli, aynı mesajları gönderebilmelidir. Bunun için ortak bir soyut sınıf ya da arayüz gerekir ve istekçi (document editor) bütün bileşenlerle bu arayüz üzerinden konuşur. O anda vekille mi resimle mi konuştuğunu bilmez; metot çağrıları şeffaf biçimde bu arayüz üzerinden yürür ve ilk karşılayan vekil olur.

Örnek çözüm (ekrandan, 01:10:00):

```
DocumentEditor --> Graphic (soyut)
                     Draw(), GetExtent(), Store(), Load()
                       ^
             +--------+---------+
            Image               ImageProxy  --image--> Image
              Draw()              Draw()
              GetExtent()         GetExtent()
              Store()             Store()
              Load()              Load()
              imageImp            fileName
              extent              extent
```

`ImageProxy.Draw()` için not:

```
if (image == 0) {
    image = LoadImage(fileName);
}
image->Draw();
```

`ImageProxy.GetExtent()` için not:

```
if (image == 0) {
    return extent;
} else {
    return image->GetExtent();
}
```

Vekil önce devreye girer, gerektiği zaman resmi yükler, sonra asıl işi resme yaptırır; kendisi aracıdır. `Draw()` içinde resim yüklenmemişse önce yüklenir, sonra çizilir; yüklenmişse tekrar yüklenmez. `GetExtent()` içinde resim henüz yüklenmemişse (işaretçi null ise) vekilin kendi tuttuğu boyut (extent) döndürülür. Hoca bunun iki biçimde olabileceğini söyledi: yüklenmemiş resmin yerinde küçük bir varsayılan resim (bir dikdörtgen içinde kırmızı bir daire, yeşil bir üçgen, mavi bir kare gibi) gösteriliyorsa onun boyutu; ya da resmin sayfada kaplayacağı en ve boy, ki metin oluşturulurken bu bilgi vekile verilebilir, vekilin resmi yükleyip öğrenmesi gerekmez. Resim zaten yüklenmişse boyut resmin kendisinden alınır. Kısacası vekil, nesne yüklenmemişse onu yüklemekten de sorumludur.

4.3 Kalıp yapısı ve Decorator ile farkı

Genel kalıp yapısı (ekrandan, 01:11:40):

```
Client --> Subject (soyut)
             Request()
               ^
       +-------+-------+
   RealSubject       Proxy  --realSubject--> RealSubject
     Request()         Request()   // ... realSubject->Request(); ...

Çalışma anında nesneler:
aClient (subject = aProxy) --> aProxy (realSubject = aRealSubject) --> aRealSubject
```

İstemci asıl nesneyle (real subject) konuşacaktır, ama farkında olmadan bir vekil üzerinden de konuşuyor olabilir. Bu yüzden ikisinin de ortak bir arayüzü olmalıdır. Vekil işi asıl nesneye yaptırır; bunu yaparken bir nevi Decorator gibi işin başına ya da sonuna ek işler koyabilir.

Hocaya göre Proxy'nin Decorator'dan önemli farkı şudur: Decorator'da üç, dört, beş, altı öğeli zincirler çok rahat kurulur ve bu doğal bir ihtiyaçtır. Proxy'de ise istemci sayılmazsa yalnız vekil ve asıl nesneden oluşan iki öğeli bir zincir vardır. Vekil yalnız o nesneye erişimi, herhangi bir isteğin başına ve sonuna yazılan ek komutlarla denetler.

4.4 Ne zaman kullanılır?

Bazı durumlarda bir nesneye basit bir işaretçi işi görebilir. Proxy, bundan daha fazlasına ihtiyaç duyulduğunda kullanılır; gerekli ek yetenekler vekilin üstüne kodlanır.

Bir kullanım yeri, vekil ile asıl nesnenin farklı bellek uzaylarında bulunmasıdır. Dağıtılmış sistemler düşünülsün: nesne ağdaki bir düğümde durur, ama ona tamamen başka bir bilgisayardan erişilmesi gerekir. Hoca dağıtılmış sistemler konusuna girmedi, çünkü bu başlı başına bir uzmanlık alanıdır. Diğer makinedeki bir vekil, sanki nesne ağdaki başka bir düğümde değilmiş gibi ona ihtiyaç duyan bilgisayara hizmet sunar: nesne neredeyse ağ üzerinden bağlantı kurar, gerekli bilgileri alır ve kendisiyle aynı bilgisayarda çalışan istemciye iletir. İstemci ek bir kodlamaya gerek duymadan nesneyi vekil üzerinden elde etmiş olur.

Nesne bir bilgisayarda dursa da erişimi hızlandırmak için ağdaki birkaç düğüme kopyalanabilir. Kopyalama varsa tutarlılık denetimi de yapılmalıdır: asıl nesnede bir değişiklik olunca bütün kopyalar güncellenmelidir; burada örneğin Observer kalıbı kullanılabilir. İstemci kopyalama, tutarlılık denetimi, önbellek (cache) ve yenileme (refresh) gibi ayrıntılarla uğraşmadan veriyi yalnız vekilden ister; bütün karmaşıklık vekilde kodlanmıştır.

Java'da bu işe yarayan RMI (Remote Method Invocation) adlı bir özellik vardır. Bir bilgisayardaki sanal makinede çalışan bir nesneye, başka bir bilgisayardaki sanal makineden erişmeyi sağlar. Sıfırdan başlamayı gerektirmeyen, dille birlikte gelen bir altyapıdır. Hoca bugün daha yetenekli araçlar olabileceğini, mikroservislerin ve bulut yapılarının da dağıtılmış sistemlerin kökenlerini taşıdığını ve oradaki iyi deneyimlerin üzerine genişletildiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** GoF kitabı büyük olasılıkla Proxy'nin başlıca türlerini uzak vekil (remote proxy: farklı adres uzayındaki nesneyi temsil eder), sanal vekil (virtual proxy: pahalı nesneyi gerektiğinde oluşturur; resim vekili bunun örneğidir), koruma vekili (protection proxy: erişim haklarını denetler) ve akıllı referans (smart reference: erişimde ek işler yapar) olarak sayar.

5. Proxy Örneği: İzle-Öde Yayın Sistemi

5.1 Problem

Bir izle-öde (pay-per-view) sistemi kurulacaktır. Dijital yayın platformları ve Netflix, Amazon Prime Video gibi çok sayıda yayın akışı (streaming) hizmeti vardır. Bunlar çevrimiçi yaygınlaşmadan önce büyük otellerde standart televizyon kanallarına ek olarak, ücreti oda faturasına yansıtılan filmler sunuluyordu. Bu hizmet niş otel ortamından evlere kadar geldi ve pandemiyle iyice yaygınlaştı. Bazı platformların kendi ek kanallarının yanında ek ücretle izlenebilen "izle-öde" filmleri de vardır.

Bu işin altyapısı için kod hazırlanıyor, ama sıfırdan başlanmıyor: elde herhangi bir videoyu herhangi bir hedefe gönderecek (stream edecek) hazır bir sunucu vardır. Bu sunucu yayını ücretsiz yapar; istenen ise yayının ek ücret karşılığında yapılmasıdır. O zaman araya bir engel konur: bu engel önce ödeme ister ya da ödemenin yapılıp yapılmadığına bakar. Ödeme yapılmışsa, müşterinin hesabında yeterli bakiye varsa, içeriği yayın sunucusundan alır ve iletir. Bu davranış tamamen bir vekil mantığıdır. Müşterinin set üstü kutusu (set-top box) doğrudan yayın sunucusuyla değil, yazılan vekil üzerinden konuşur; vekil gerekli ödeme denetimini yapar ve yeterli bakiye varsa içeriği sunucudan alıp istekçiye iletir.

Bu mantığa koruma vekili (protection proxy) denir. Literatürde başka vekil tanımları da vardır; hepsi bu kalıpla yapılabilir, çünkü hepsi aynı noktaya dayanır: asıl işlemin başına ve sonuna eklenen metot çağrıları.

5.2 Yapı

Çözüm (ekrandan, 01:20:00):

```
«interface» StreamSource
  + stream(programID, destination : String) : boolean
      ^ (gerçekler)                ^ (gerçekler)
  StreamServer                  StreamProxy
    + stream(...) : boolean        - original : StreamSource      (1:1)
                                   + StreamProxy(original : StreamSource)
                                   + stream(programID, destination : String) : boolean
                                   + checkPayment(programID, destination : String) : boolean

StreamDestination  --server (1:1)--> StreamSource
  - server : StreamSource
  - address : String
  + StreamDestination(address : String)
  + getAddress() : String
  + setServer(StreamSource)
  + makePayment(double) : boolean
  + requestProgram(programID : String) : boolean

MainApp --> StreamDestination, StreamServer, StreamProxy
```

Elde zaten olan yayın sunucusu mutlaka bir arayüz sunuyordur: `StreamSource`. Bu arayüzde, belli bir programı belli bir hedefe gönderebiliyorsa `true`, gönderemiyorsa `false` döndüren `stream` metodu vardır. Yazılacak yeni sınıf `StreamProxy` de bu arayüzü gerçekler. Vekili olacağı nesneyi `original` adlı bir üyede tutar. Bu aslında bir `StreamServer` ile birebir sahipliktir, ama üye `StreamSource` arayüzü üzerinden erişildiği için en doğru gösterim şemadaki gibidir. Vekil, asıl sunucuyu kurucuda alır.

Vekil aynı imzalara sahip olmak zorunda olduğu için asıl işlem olan `stream` metodunu mutlaka gerçekler. Ek olarak `checkPayment` metodu vardır: belli bir programı belli bir hedefe göndermek için o hedefin hesabında yeterli para var mı diye bakar. Yoksa ekrana "lütfen ödeme yapın" gibi bir diyalog çıkaran bir gövde yazılabilir.

`StreamDestination` sınıfının adresini alma, sunucu atama, ödeme yapma ve belli bir programı isteme metotları vardır. Her hedef doğrudan `StreamServer` ile değil, `StreamProxy` nesneleriyle ilişkilendirilir. Hedef, sunucuyla mı vekille mi konuştuğunu bilmez, bilemez ve bilmek zorunda da değildir. Örnek bir ana programda hedefler ve vekiller oluşturulup ilgili hedeflere ilgili vekiller atanır.

5.3 Kod

Kaynak kodlar (`dp.proxy.example`) çok zor değildir. `StreamSource` arayüzü `stream` metoduna sahiptir; `StreamServer` bu metodu gerçekler, ama gövdesi yazılmamıştır. `StreamProxy` (ekrandan, 01:23:10):

```
private StreamSource original;

public StreamProxy(StreamSource original) {
    this.original = original;
}

public boolean stream(String programID, String destination) {
    if (checkPayment(programID, destination))
        return original.stream(programID, destination);
    else
        return false;
}

public boolean checkPayment(String programID, String destinatio...
    // ödemeyi destination parametresine göre kontrol et
```

Vekilin `stream` metodu, asıl `stream` çağrısından önce `checkPayment` çalıştırır. Başarılıysa vekili olduğu nesnenin `stream` metodunu kullanıp sonucu istekçiye iletir; aksi hâlde işi yapmaz. `checkPayment` nasıl kodlanması gerekiyorsa öyle kodlanır; örnekte sabit bir değer döndürür.

İlişkiler şöyle kurulur: istemciye sunucu verilecekse asıl sunucu değil vekil verilir. Vekile asıl sunucu, istemciye vekil verilir. Hoca kalıbın karmaşık olmadığını söyledi.

6. Davranışsal Kalıplara Geçiş ve Chain of Responsibility

6.1 Davranışsal kalıplar

Proxy ile yapısal kalıplar bitti ve davranışsal (behavioral) kalıplara geçildi. Hoca o ana kadar on üç kadar kalıp üzerinde çalışıldığını, yaklaşık on bir kalıp kaldığını söyledi.

> **Ders sonrası not:** GoF kataloğu büyük olasılıkla 23 kalıptan oluşur: 5 yaratımsal, 7 yapısal ve 11 davranışsal. Yaratımsal ve yapısal kalıplar toplam 12 eder; hocanın "on üç" demesi büyük olasılıkla yaklaşık bir sayımdır. Kalan davranışsal kalıp sayısı (11) doğrudur.

6.2 Amaç

İlk davranışsal kalıp Chain of Responsibility (Sorumluluk Zinciri) kalıbıdır. Hoca kalıbın adının, kalıp anlaşıldıkça ne kadar yerinde olduğunun görüleceğini söyledi.

Elde zaten ilişkili bir nesneler zinciri ya da hiyerarşisi vardır. Bu, Composite'teki gibi bir yapı olabilir, ama illa Composite olması gerekmez: örneğin bir çerçeve (frame), içinde paneller ve panellerin içinde çeşitli GUI bileşenleri. Bu nesnelerden birinin bir metodu çağrıldığında, o nesne metodu anlamakla birlikte en iyi, en doğru ya da en ayrıntılı yanıtı veremeyebilir. Kalıbın amacı, bu durumda mesajın, gereğini daha iyi yerine getirebilecek başka bir nesneye iletilmesidir (ekrandan, 01:30:00: "Birbirleri ile ilişkili bir nesneler zinciri veya hiyerarşisindeki bir nesnenin, aldığı bir mesajı işleyemeyeceği durumlarda, bu mesajın gerekli işlemi yapabilecek bir başka n[esneye iletilmesi]"). Amaç, o nesnede o metodun olmaması değil, o nesnenin metodun gereğini daha iyi yerine getirebilecek bir nesneyi bilmesidir. Olmayan bir metodu kazandırmak gibi bir şey kesinlikle söz konusu değildir.

Hoca burada Smalltalk'u andı. Smalltalk ilk nesneye dayalı dillerden biridir ve ilginç bir özelliği vardır: bir sınıfın olmayan bir metodu çağrıldığında derleme hatası vermez. Nesneye tanımlı olmayan bir metot çağrılmışsa, nesnede tanımlı `doesNotUnderstand` adlı bir metot devreye girer. Hoca bunun ilginç bir yaklaşım olduğunu, ama Chain of Responsibility'nin bunun üzerine kurulu olmadığını vurguladı: zincirdeki nesnede metot vardır, tanımlıdır; yalnız en doğru hesaplamayı o değil, ilişkide olduğu başka bir nesne yapabilir.

6.3 Bağlama duyarlı yardım örneği

Örnek bir diyalog kutusudur; Office'in eski sürümlerinde çok kullanılan, yenilerinde pek yapılmayan bir örnektir. Hoca derste PowerPoint'in Yazı Tipi diyaloğunu açtı. Diyelim ki "Üst simge" seçeneğinin ne olduğu merak edildi. Diyaloğun köşesindeki soru işaretine tıklayınca bugün genel yardım açılır. Eskiden böyle değildi: soru işaretine tıklanınca fare imlecinin yanına bir soru işareti eklenir, sonra hangi bileşene tıklanırsa özellikle o bileşenle ilgili yardım, belki bir açılır pencere olarak, belki yardım dosyasının ilgili yerine gidilerek gösterilirdi. Hoca bu özelliği çok sevdiğini söyledi: koskoca bir yardım içinde arama yapmaktansa odaklanmış yardım gelir. Bu tür bir özelliği kodlamak için Chain of Responsibility çok yararlı bir kalıptır.

Diyelim ki programcı o kadar ayrıntılı bir yardım dosyası hazırlayamadı. Yazı tipi sekmesi için genel bir yardım içeriği ve alttaki efektler kısmı için genel bir yardım içeriği var, ama efektler panelindeki tek tek onay kutuları için özel bir yardım yok. Diyalog kurulurken yazı tipi sekmesi oluşturulmuş, içine efektler paneli eklenmiş, panelin içine de etiketleriyle birlikte onay kutuları eklenmiştir. Bu bileşenlerin her biri için ayrı yardım yazılamasa da, onay kutusuna soru işaretiyle tıklanınca en azından efektler kısmını toplu anlatan yardım çıkabilir.

Sonraki slaytta nesnelerin ilişkisi görülür. Bir yazdırma diyaloğunda (print dialog) "Tamam" (OK) ve "Yazdır" (Print) düğmeleri vardır. Her düğme hangi diyalog kutusunun içinde olduğunu bilir. Uygulamada başka diyaloglar da vardır ve hepsi uygulamanın içindedir. Özelden

genele doğru bir silsile vardır; en genel olan uygulamanın kendisidir. Yardım isteği OK düğmesine iletildiyse ve düğmeye ayrıntılı bir yardım bilgisi bağlanmışsa onu gösterir. Yoksa ilişkili olduğu yazdırma diyaloğuna "yardım isteği geldi, karşılayabilir misin?" diye sorar. Diyalog karşılayabiliyorsa içinde o düğmenin bilgisi de bulunan kendi yardımını gösterir. Onda da yoksa uygulamanın ana yardımı çıkar. OK düğmesinde yardım metodu vardır, ama içeriği kodlanmamışsa çağrıyı bir üst bileşene yönlendirir. Bütün varlıklar doğrudan zincir biçiminde ilişkili olmasa da aralarında bir zincir kurulabilir. Slaytta bu akış bir sıralama diyagramıyla (sequence diagram) da gösterilmiştir.

6.4 Çözümün sınıf yapısı

Bunun için bütün bileşenlerin aynı metodu yanıtlayabilmesi, yani yardımla ilgili ortak bir arayüz gerekir (ekrandan, 01:34:10):

```
HelpHandler (soyut)  --handler--> HelpHandler   (kendine ilişki)
  HandleHelp()          // varsayılan: handler->HandleHelp()
    ^
    +-- Application
    +-- Widget (soyut)
          ^
          +-- Dialog
          +-- Button
                HandleHelp()   // if can handle { ShowHelp() }
                               // else { Handler::HandleHelp() }
                ShowHelp()
```

`HelpHandler` soyut bir sınıftır ve `HandleHelp` soyut bir metottur. Aradaki zincir ilişkisini kurmak için sınıfın kendinden kendine birebir bir sahiplik ilişkisi vardır. Alt sınıfları uygulama ve çeşitli bileşenlerdir (widget); `Widget`'ın alt sınıfları da diyalog kutuları, düğmeler ve her türlü GUI bileşenidir. Örneğin `Button.HandleHelp()`: düğmeye özel bir yardım yüklenmişse onu gösterir, aksi hâlde isteği ilişkili olduğu üst makamın `HandleHelp` metoduna iletir.

6.5 Kalıp yapısı ve tasarım ilkesi

Kalıp biçimine getirildiğinde bir `Handler` arayüzü (ya da soyut sınıfı) ve somut handler'lar olur. Aralarında mutlaka bir ilişki vardır. Nesneler arasında zaten bir ilişki varsa bu ilişki yeniden konmak zorunda değildir; nesnelerin kendi içinde de yer alıyor olabilir. Ama en doğrusu bu ilişkinin ortak bir üst sınıfta ya da arayüzde olmasıdır. Aksi hâlde örneğin `ConcreteHandler2`, kendisinin `ConcreteHandler1` ile mi yoksa `ConcreteHandler3` ile mi ilişkili olduğuna sınıf adlarına bakarak karar vermeye çalışır; bu da if-else-else zincirleri getirir ve doğru değildir.

> **Hocanın vurgusu:** İlişkinin kurulması ve "bir üst makam kim" sorusunun yanıtlanması işleri, bütün alt sınıflarda ortak olarak yapılacağı için soyut üst sınıfa eklenecek somut metotlarla kodlanmalıdır.

6.6 Ne zaman kullanılır? Zincir uyarısı

Kalıp, bir mesajı yanıtlayabilecek birden fazla nesne bulunduğunda ve bunlardan hangisinin yanıt vermesinin en doğru olacağı önceden belirlenemediğinde ya da çalışma anında belirlenmesi tercih edildiğinde kullanılır.

> **Hocanın vurgusu:** Bu kalıbı kullanırken zincirin kırılmamasına, doğru yapılandırılmasına ve doğru sonlandırılmasına çok dikkat edilmelidir.

Aksi hâlde bir metot çağrısı boşluğa düşebilir. Düğme örneğinde yardım mesajı düğmeye geldi, düğme yapamadı, yazdırma diyaloğuna iletti; o da yapamadı, uygulamaya yönlendirdi. Uygulama da yapamıyorsa gönderebileceği başka bir yer yoktur. O zaman hiçbir tepki verilmeyebilir. Bu durumda en azından uygulama "özür dileriz, yardım dosyası bulunamıyor" demelidir. Yoksa kullanıcı tıklar, bir şey olmaz ve "program mı takıldı, bilgisayar mı takıldı?" diye düşünür. Hazırda bir zincir yoksa onu oluşturma işi de az önce söylendiği gibi soyut Handler sınıfına eklenebilir.

7. Chain of Responsibility Örneği: Reklam Yönetim Yazılımı

7.1 Problem

Örnek hocanın kendi örneğidir (slaytta "Kaynak: YES"). Bir reklam yönetim yazılımı hazırlanmaktadır. Elde bir web sitesi kataloğu vardır: biri spor haberleri veren bir site, biri genel haber sitesi gibi. Farklı siteler farklı kullanıcı kesimlerine hitap eder. Bu sitelere çeşitli müşterilerden reklam alınır. Müşterinin reklamının hedef kitlesi ile elimizdeki sitelerin ziyaretçileri arasında bir uyuşma varsa bundan yararlanmak istenir; böylece reklamlar hedeflenen kişilere daha iyi ulaşır.

İlgisiz reklamlara kızılır, "saçma reklam" denir. Ama spor sitesinde haber okuyan birinin, spor malzemeleri satan bir mağazadaki indirimin reklamını görmesi hoşuna gidebilir ve belki alışveriş bile yapar. Ayrıca her müşteri reklamının belli sayıda gösterilmesini ister ve buna göre ödeme yapar; hedefe ulaşılamazsa paranın bir kısmı iade edilebilir. Yani web sitesi havuzu ile reklam kampanyaları havuzu arasında en iyi eşleştirme yapılmaya çalışılır. Bu altyapı kurulacaktır.

Kampanyalar (ekrandan, 01:40:00):

- Kampanya 1: Spor sitesinin 18-39 yaş arası erkek ziyaretçilerine traş bıçağı reklamı göster, 18-39 yaş arası kadın ziyaretçilerine fitness salonu reklamı göster, diğer ziyaretçilere spor mağazasındaki indirimin reklamını göster.
- Kampanya 2: Haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamını göster, aksi hâlde spor sitesinin kampanyasını aynen uygula.
- Kampanya 3: Ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından birini göster.

Reklam gösterme metodu çağrılırken elde bir web sitesi, onun bir ziyaretçisi ve ziyaretçinin demografik bilgileri vardır. Hoca bu bilgilerin nasıl elde edildiğinin sorulmamasını istedi; en masum yol, üyelik formunda yaş, cinsiyet ve gelir düzeyi sormak, bazı alanları zorunlu tutmaktır. Elde net bilgi varsa karar ona göre verilir.

7.2 Neden Chain of Responsibility?

Elde bir ziyaretçi ve bir dizi reklam vardır. Reklamlara tek tek bakılır: bu reklam bu ziyaretçiyle uyuşuyor mu? Uyuşmuyor, sonrakine bakılır. O uyuşuyor mu? Uyuşmuyor, sonrakine. Bu uyuşuyor: o gösterilir. Ayrıca bir kampanyanın başka kampanyaları sonuna ya da başına ekleyebilmesi gerekir (kampanya 2, kampanya 1'i içerir). Bir koleksiyondan uygun olanı göstermek ve kampanyaları birbirine eklemek düşünüldüğünde Chain of Responsibility mantıklı bir kalıp olarak görünür.

7.3 Sınıflar

Kullanıcı sınıfı `User` cinsiyet, gelir düzeyi ve yaş tutar. Cinsiyet bir enum'dır (kadın, erkek); gelir düzeyi de `A`, `B`, `C` değerli bir enum'dır; yaş bir tam sayıdır. Kurucu ve bütün alanlar için getter'lar vardır.

Kampanyalarda belli bir yaş değil, yaş aralığı söz konusudur. Hoca, bir kullanıcının belli bir yaş aralığında olup olmadığını denetleme işini `User` sınıfına koydu, çünkü bu işin sürekli tekrarlanacağını gördü ve "bu bilgi kimde? User'da; o zaman metot ona eklensin" diye düşündü. `yasDahilMi(alt, ust)` metodu, alt ve üst sınır dahil olmak üzere kullanıcı o aralıktaysa `true`, değilse `false` döndürür.

Reklamlar soyut `Reklam` sınıfından türer. Her reklam bir banner (bir bitmap) gösterir; bu kodlanmadı, `Banner` sınıfında boş gövdeli bir `show` metodu vardır, reklam yalnız kendi adını yazar. Kampanyalar düzenleneceği için bir reklam bir sonraki reklamla ilişkili olabilir; bu yüzden `sonraki` reklam bir üye alandır. Hoca şemada bu kendinden kendine ilişkinin ok olarak çizilmediğini fark etti: şemaları Eclipse'e kurduğu bir UML eklentisiyle büyük ölçüde otomatik oluşturduğunu, bir seçeneği tıklamayı atlamış olabileceğini söyledi.

`Reklam` soyut bir sınıftır: kurucusu somut kodlanabilir, ama belirli bir reklamın belirli bir kullanıcıya uygun olup olmadığı burada kodlanamaz, çünkü reklamların kuralları çok farklıdır.

Reklamı göstermek de türüne göre farklı şeyler gerektirebilir: bir bitmap, bir HTML5 animasyonu ya da bir video yüklemek. Bu yüzden `reklamGoster` de soyut yapıldı. `reklamUygunMu` ise zaten mutlaka soyut olmalıdır. Zinciri izlemeyi ve yürütmeyi sağlamak için bir de `reklamKotar` metodu hazırlandı: reklam uygunsa göster, değilse sonraki reklamın `reklamKotar` metodunu çağır. İlişki `sonraki` ile kuruldu; ilişkinin izlenmesi ve yürütülmesi bütün reklamlarda ortak olduğu için bu metoda kondu.

Her reklam, adından ne olduğu anlaşılan bir alt sınıfa kodlanır; asıl iş, `reklamUygunMu` metodlarına yazılan koddur. Başka reklamlar gerekirse yeni alt sınıflar eklenir; yalnız o sınıfı kodlamak yeter. Sonrakiyle ilişki üst sınıftan hazır gelir, alt sınıfa yalnız uygunluk kuralı yazılır; başka yerlerde değişiklik yapılmaz. Yeni bir demografik bilgi gerekirse o da yeni bir ihtiyaçtır ve ilgili yere eklenerek halledilir.

7.4 Kod

Proje paketi `dp.chainOfResponsibility.example` içinde `Banner`, `Cinsiyet`, `FitnessSalonu`, `GelirDuzeyi`, `IndirimSporMagaza`, `MainApp`, `OtoDergi`, `Reklam`, `TrasBicak` ve `User` sınıfları vardır. `Cinsiyet` enum'ı (ekrandan, 01:50:00):

```
public enum Cinsiyet {
    KADIN("Kadın"), ERKEK("Erkek");
    private String tanim;
    private Cinsiyet(String tanim) { this.tanim = tanim; }
    public String toString() { return tanim; }
}
```

`GelirDuzeyi` yalnız değerlerden oluşan basit (ilkel) bir enum'dır. `Cinsiyet` ise daha karmaşık bir enum'dır: Java'da enum değerlerinin yanında kurucu parametreleri verilebilir, üye alanlar ve üye metotlar tanımlanabilir.

Soyut `Reklam` sınıfı (ekrandan, 01:51:10; ekranın altı kesik):

```
public Reklam(Banner banner, Reklam sonraki) {
    this.banner = banner; this.sonraki = sonraki;
}
public abstract boolean reklamUygunMu(User u);
public abstract void reklamGoster(User u);
public final void reklamKotar(User u) {
    if (reklamUygunMu(u)) {
        System.out.println(this.getClass().getName()
            + "->" + u);
        reklamGoster(u);
    }
    else if (sonraki != null) {
        System.out.println(this.getClass().getName()
            + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
}
```

Kurucu banner'ı ve sonraki reklamı atar. `reklamKotar` önce bu reklamın bu kullanıcıya uygun olup olmadığına bakar; ilişki soyut bir metot üzerinden kurulur. Uygunsa reklam gösterilir. `reklamGoster` gövdeleri boştur; zincirin doğru çalıştığını komut satırında görmek için sınıf adı, bir ok ve kullanıcı yazdırılır: bu, "reklam bu kullanıcıya gösterildi" anlamına gelir. Uygun değilse reklam kendini göstermez, kotarma çağrısını sonraki reklama iletir ve "bu reklam sınıfı -> sonraki reklam sınıfı" biçiminde bir iz yazar.

Zincir doğru sonlandırılmazsa kullanıcıya hiçbir reklam gösterilmeyebilir. Bu yüzden somut reklamlardan biri, `reklamUygunMu` metodu her zaman `true` döndüren, her kullanıcıya gösterilebilen bir reklamdır ve zincirin sonuna konur. Zincirin doğru kurulup kurulmadığını burada denetlemek çok uygun değildir. Son bir `else` konup "ne olursa olsun göster" mantığı buraya da yazılabilirdi, ama hoca bu ayrıntıyı oluşturulacak reklamlarda tanımlamayı, kararı üst sınıfa vermemeyi kesinlikle tercih ettiğini söyledi; reklamlara genel değil özel olarak yaklaşmak

istedi. En fazla buraya bir `else` konup bir çalışma zamanı istisnası (runtime exception) fırlatılabilirdi, ama bunu da yapmadı.

`IndirimSporMagaza` her tür kullanıcıya uygundur, çünkü kampanyada "diğer ziyaretçiler kim olursa olsun spor mağazası indirimini göster" denmiştir.

> **Hocanın vurgusu:** Spor mağazası indirim reklamının `reklamUygunMu` metoduna yazılan `return true` bir iskelet kod değil, iş mantığının kendisidir. Bu reklam zincirin sonuna konunca zincir doğru sonlandırılmış olur ve en kötü olasılıkta bile bir reklam gösterilebilir; hiç reklam gösterilemezse para da alınamaz.

`TrasBicak` (ekrandan, 01:55:30):

```
public void reklamGoster(User u) {
    //Gerekli komutlar
}

public boolean reklamUygunMu(User u) {
    if (u.getCinsiyet() == Cinsiyet.ERKEK)
        if (u.yasDahilMi(18, 39))
            return true;
    return false;
}
```

Kampanyadaki "18-39 yaş arası erkek" kuralı böyle kodlanır: cinsiyet erkekse ve yaş 18-39 aralığındaysa `true`, aksi hâlde `false`. Bu sayede yalnız ilgili kural kodlanır, başka hiçbir şeyle uğraşılmaz; altyapı üst sınıftan gelen somut metotlarla hazırdır. `OtoDergi` da benzer biçimde "erkek, A gelir grubu, 18-29 yaş" kuralını kodlar.

7.5 Ana program ve çalıştırma

Ana programda (ekrandan, 02:00:00) önce kullanıcılar oluşturulur, sonra kampanya 1'in zinciri kurulur:

```
users = new User[3];
users[0] = new User(Cinsiyet.ERKEK, GelirDuzeyi.C, 23);
users[1] = new User(Cinsiyet.ERKEK, GelirDuzeyi.A, 53);
users[2] = new User(Cinsiyet.KADIN, GelirDuzeyi.B, 33);

public void kampanya1() {
    Reklam[] reklamlar = new Reklam[3];
    reklamlar[2] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2]);
    reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}
```

Spor mağazası indirimi en başa konursa her kullanıcıya hemen o gösterilir; bu yüzden önce traş bıçağı, sonra fitness salonu reklamı gelmelidir. Hoca diziyi ters sırada doldurmuştu ve bunun nedenini önce hatırlayamadı. Sonra açıkladı: spor mağazası reklamının sonrası yoktur, bu yüzden önce o oluşturulur; fitness salonu oluşturulurken sonraki reklamı dizide hazırdır; traş bıçağı oluşturulurken de öyle. Ters sırada koymak da sorun değildir, doğru çalışır. Sonra sırayla üç kullanıcıya reklam gösterilmesi istenir: kampanya kurallarına uygun reklamlar mı gösterilecek, yoksa bir kadın ziyaretçiye traş bıçağı reklamı gibi anlamsız bir şey mi?

Çıktı (ekrandan, 02:00:00; her satırın başındaki `dp.chainOfResponsibility.example.` paket adı kısaltıldı):

```
TrasBicak->Erkek,C,23
TrasBicak->FitnessSalonu
FitnessSalonu->IndirimSporMagaza
IndirimSporMagaza->Erkek,A,53
```

```
TrasBicak->FitnessSalonu
FitnessSalonu->Kadın,B,33
```

İlk kullanıcı C gelir düzeyinde 23 yaşında bir erkektir. Zincirin başında traş bıçağı vardır ve kural “18-39 yaş erkek” der; gelir grubu önemli değildir. Bu reklam uygundur, gösterilir ve iş biter; başka reklamlar arasında dolaşılmaz.

İkinci kullanıcı A gelir grubunda ama 53 yaşında bir erkektir. Traş bıçağı reklamı devreye girer, ama 39 yaşa kadar olan erkekleri hedeflediği için kendini göstermez ve sonrakine, fitness salonuna iletir. Fitness salonu kadın ziyaretçi istediği için o da iletir. Spor mağazası indirimi kim olursa olsun gösterilir ve bu kullanıcıya o gösterilir.

Üçüncü kullanıcı 33 yaşında bir kadındır. Traş bıçağı erkek aradığı için iletir. Fitness salonu 18-39 yaş arası kadın ziyaretçiyi aradığı ve kullanıcı bu aralıkta olduğu için kendini gösterir.

Hoca, yeni reklamlar, yeni kurallar ya da yeni kullanıcılar tanımlayıp kodu inceleyerek bir hata olup olmadığına kolayca bakılabileceğini söyledi. Kampanya 2 de benzer biçimde yapılabilir: işin içine otomobil dergisi reklamı girer. Hoca bunu derste çalıştırmadı, ama örneğin doğru çalıştığını, öğrencilerin de çalıştırıp görmesini söyledi.

8. Command Kalıbı

8.1 Amaç

Kısa bir aradan sonra dersin son kalıbı olan Command’a geçildi. Kalıbın amacı, nesneye giden istekleri, yani metot çağrılarını, birer nesne hâline çevirmektir. Metotları nesne yapmaya çalışmak ilk bakışta ilginç gelir; acaba işe yarar mı? Yarar: örneğin metot çağrıları kuyruğa alınabilir ya da günlüğü (log) tutulabilir.

8.2 GUI menüsü örneği

GUI programlarında bir dosya menüsünün altına çeşitli menü seçenekleri ya da alt menüler kodlanır ve bir seçeneğe tıklanınca bir işin yapılması istenir. Bu sürekli yapılan bir şeydir. Kullanılan çatılara (framework) bakılırsa aslında Command kalıbını gerçekledikleri görülür. Uygulama bir menü, menü de menü öğeleri (menu item) ya da alt menüler içerir.

Bir öğenin tıklanma metodu içine doğrudan bir şeyler kodlamak yerine bir komut sınıfı olması ve tıklanınca bu komutun `execute` metodunun çalışması istenir. Böylece menü öğesi, düğme, radyo düğmesi gibi bileşenlere, tıklandıklarında, `Command` arayüzünü gerçekleyen ve `execute` metoduna sahip herhangi bir nesne verilerek o nesnenin işi yaptırılabilir. Yani belirli bir öğenin tıklanma olayına kod yazmak yerine bir soyutlama adımı daha eklenir: bir `Command` arayüzü ya da soyut sınıfı ve onun `execute` metodu.

Bu tasarımda örneğin bir menü seçeneğiyle bir dosya açma komutu ilişkilendirilir. Komutlardan biri `OpenCommand` olur; `execute` metodu ve başka metotları da olabilir. `execute` gövdesi istenen biçimde kodlanarak dosya açma yapılır; açılan dosya uygulamadaki açık belgeler arasına eklenebilir. Komut, kendisini çağıran nesnenin (uygulamanın) bir metodunu da çağırabilir. Benzer biçimde uygulama farklı türden belgeler işliyor ve bunlardan birine kopyalanan bir şey yapıştırılacaksa, yapıştırma da `Command`’ın bir alt sınıfı olur ve `execute` içinde yapıştırma işlemi belgeye nasıl yapılıyorsa öyle kodlanır.

8.3 Kalıp yapısı

Kalıp yapısı (ekrandan, 02:20:00):

```
Client  --> Receiver
Client  ..> ConcreteCommand (oluşturur)
Invoker <>--> Command (soyut)
                Execute()
                  ^
            ConcreteCommand  --receiver--> Receiver
              Execute()   // receiver->Action();      Action()
              state
```

Bir istemci (client) vardır. Asıl komutları çalıştıran varlık çağırıcıdır (invoker). Her komutun bir alıcısı (receiver) olabilir de olmayabilir de: belki çağırıcı bir komutu basitçe yürütür ve o komutun özel bir alıcı nesnesi yoktur. Yani alıcı kısmı isteğe bağlıdır. Ama alıcı desteğinin kodlanması yararlıdır; şu anda gerekmese bile, belirli bir alıcıya sahip olması gereken komutların yakında kodlanması gerekebilir. Gereksinim belgesinde öyle yazıyorsa kodlanır. Şart değildir, ama alıcı, kalıbın önemli bir yeteneğidir.

Yürütme metodu arayüz olarak tanımlanır. Somut gerçeklemelerde komut nasıl yürütülecekse öyle yürütülür: komutun bir alıcısı varsa onun `Action` metodu çağrılır, ya da sınıf işi yürütmek için yalnız kendi metotlarına dayanır. Neye ihtiyaç varsa ona göre kodlanır.

Hocaya göre kalıbın en güzel örneği GoF'un verdiği dosya menüsü örneğidir. Kendi örneğinden çok memnun olmadığını, daha iyi bir örnek yapılabileceğini, ama aklına gelmediğini ve hazır kaynaklarda da çok güzel bir örnek bulamadığını söyledi. Öğrenciler ek kaynaklarda hoşlarına giden bir Command örneği bulurlarsa paylaşmalarını rica etti.

9. Command Örneği: Akıllı Ev Kumandası

9.1 Problem ve yapı

Örnek Head First Design Patterns kitabındaki örneğin biraz değiştirilmiş hâlidir. Bir akıllı ev yazılımı üretilmektedir. Yazılımda basit bir açma/kapama kumandası ekranı vardır. Kumanda ekranındaki aç/kapa düğmelerine odaların ışıklarını, televizyonları ve benzerlerini açma/kapama işlevleri kazandırılacaktır (ekrandan, 02:23:40). Belki çeşitli senaryolar tanımlanabilecek, daha karmaşık komutlar verilebilecektir.

Yapı (ekrandan, 02:23:40):

```
«interface» AcKapa
  + ac() : boolean
  + kapa() : boolean
     ^ (gerçekler)
     +-- AydinlatmaDugmesi  (- isim : String; + ac(), + kapa(), + toString(), + AydinlatmaDugm
     +-- ElektronikAlet     (- isim : String; + ac(), + kapa(), + ElektronikAlet(isim), + toSt
     +-- Iklimlendirme      (- isim : String; + ac(), + kapa(), + Iklimlendirme(isim), + toStr

Kumanda  --* AcKapa
  - acKapaDugmeleri : ArrayList<AcKapa>
  + ac(dugmeNo : int) : boolean
  + Kumanda()
  + dugmeEkle(dugme : AcKapa)
  + kapa(dugmeNo : int) : boolean
  + hepsiniKapa()

AnaProgram  --> Kumanda
  - kumanda : Kumanda
  + AnaProgram()
  + evBoskenSokakKapisiAcildi()
  + evdeKimseKalmadi()
  + main(args : String[])
```

Kontrol paneli `Kumanda` sınıfı olarak ele alındı. Kumandada aç-kapa düğmeleri vardır. Bir düğme iki iş yapabilir: açar ya da kapatır. Bu iki metot `AcKapa` arayüzünde tanımlıdır. Arayüzü gerçekleyen somut bileşenler bir ışık için aydınlatma düğmesi, elektronik aletleri temsil eden `ElektronikAlet` ve klima, kombi gibi ısıtma-soğutma cihazlarını temsil eden `Iklimlendirme` sınıfıdır. Isıtıcı ve soğutucu ayrı sınıflar olabilirdi, ama klimalar gibi bazı cihazlar hem ısıtır hem soğutur; yazılımın arkasındaki hikâyeye göre tek sınıf yetebilir ya da iki sınıf gerekebilir.

Cihaz sınıfları `ac` ve `kapa` metotlarını gerçekler; ne olduklarını tanıtmaları için bir de `toString` metodu vardır. Tuttukları bilgi yalnız adlarıdır. Ana program kumandayı içerir. Ayrıca "ev boşken sokak kapısı açıldığında şunlar yapılsın", "evde kimse kalmayınca bunlar yapılsın" gibi senaryolar tanımlanabilmek istenir; bu yüzden akıllı ev uygulamasının kendisi ana programdır.

Ek bilgi (derste anlatılmadı): Bu örnekte kalıbın rolleri büyük olasılıkla şöyle eşleşir: `AcKapa` arayüzü Command, `Kumanda` Invoker, `AnaProgram` Client rolündedir. Cihaz sınıfları hem ConcreteCommand hem de Receiver görevini üstlenir; yani ayrı bir alıcı nesnesi yoktur. Bu, hocanın derste belirttiği "receiver isteğe bağlıdır" durumuna uyar.

9.2 Kod

Kumanda (`dp.command.example`) aç-kapa düğmelerine sahiptir. Kurucu `ArrayList`'i ilklendirir. `dugmeEkle` basitçe listeye, ortak arayüz türünden bir eleman ekler. `ac(dugmeNo)`: ilgili numarada düğme yoksa `false` döndürür; varsa ilgili nesneyi listeden alır, `ac` metodunu çalıştırır ve o ne döndürdüyse onu döndürür. `kapa` tamamen benzerdir, yalnız açmak yerine kapatır. Ayrıca evde kimse kalmayınca gerekebilecek, her şeyi kapatan bir `hepsiniKapa` metodu eklenmiştir. Kumandaya akla gelen başka yararlı metotlar da eklenebilir.

`AydinlatmaDugmesi`, `AcKapa` arayüzünü gerçekler; "salonun elektrik düğmesi" gibi bir isim kurucuda atanır. `ac` metodu, kendisini açık konuma getirdiğini söyleyen bir yazı ekrana basar (`toString` çağrılarak "AcKapa:Salon" gibi bir ad yazılır) ve `true` döndürür. `kapa` da benzer biçimde kendi bilgisini verip `true` döndürür; hiç `false` döndürmez. `ElektronikAlet` ve `Iklimlendirme` de tamamen aynıdır; ek bir mantık kodlanmadı. Gerçekte bir iklimlendirme cihazının açılışında "önce şu çalışsın, sonra şu devreye girsin" gibi farklı adımlar olacaktır.

Ana program ilk açıldığında evdeki her şeyi sisteme ekler: antre, salon, mutfak ve diğer odanın aydınlatma düğmeleri, elektronik alet olarak bir televizyon ve iklimlendirme cihazı olarak bir kombi. Senaryolar (ekrandan, 02:30:00):

```
public void evBoskenSokakKapisiAcildi() {
    kumanda.ac(0);
    kumanda.ac(5);
}
public void evdeKimseKalmadi() {
    kumanda.hepsiniKapa();
}
public static void main(String[] args) {
    AnaProgram prg = new AnaProgram();
    prg.evBoskenSokakKapisiAcildi();
    System.out.println("Evden çıkılıyor");
    prg.evdeKimseKalmadi();
}
```

Ev boşken sokak kapısı açılınca kumandadaki 0 ve 5 numaralı elemanlar çalışır: sıra 0, 1, 2, 3, 4, 5 olduğuna göre 0 antre ışığı, 5 kombidir. Hoca kombiyi herhalde soğuk havada yazdığını, gerçekte bir sıcaklık algılayıcısı varsa onun durumuna bakılabileceğini söyledi. Bir senaryo gerçekleşince bazı bileşenlerin bazı komutlarının yürütülmesi gibi senaryolar Command kalıbıyla rahatça kodlanır. Evde kimse kalmayınca kumandanın `hepsiniKapa` metodu çağrılır.

Çalıştırınca önce antre ışığı ve kombi kendilerini açık konuma aldıklarını bildirir; sonra "Evden çıkılıyor" yazılır ve "evde kimse kalmadı" senaryosu yürütülür. Çıktının görünen kısmı (ekrandan, 02:30:00):

```
ACIK: AcKapa:Kombi
Evden çıkılıyor
KAPALI: AcKapa:Antre
KAPALI: AcKapa:Salon
KAPALI: AcKapa:Mutfak
KAPALI: AcKapa:Oda
KAPALI: AcKapa:Vestel TV
KAPALI: AcKapa:Kombi
```

Bütün bileşenler kendini kapatıp adını söyler. Hoca kalıbı ve örneği bu biçimde bitirdi; bir sonraki kalıp Iterator olacaktı, ama uzun süreceği için o derse başlanmadı.

10. Örnek Sorular

Soru 1 (derste sözlü, 00:26:24): Bir hava yolu şirketi bir ayda bir milyon bilet satmıştır. Yolcuların ortalama ayda iki kez uçtuğu varsayılırsa kaç tekil müşteri vardır? Tekil müşteriler önemliyse bu problem Flyweight ile modellenebilir mi?

> **Çözüm (derste verildi):** $1\,000\,000/2 = 500\,000$ tekil müşteri vardır (python ile doğrulandı). Tekil müşteriler önemliyse, yani her kişinin kendi kimliği ayırt edilmeliyse Flyweight kullanılamaz: Flyweight'te paylaşılan nesneler aynı nesnedir ve nesne kimliği kaybolur.

Soru 2 (derste sözlü, 00:31:34): Kimya fabrikası örneğinde maddeler ve karışımlar neden Composite kalıbıyla modellenmedi?

> **Çözüm (derste verildi):** Composite hiyerarşik bir parça-bütün ilişkisini modellemek için kullanılır. Karışımda böyle bir hiyerarşi yoktur; maddeler bir kaba aynı düzeyde eklenir. Ekleme sırası gerekiyorsa bağlı liste bunu tutar. Ara birleşimler oluşturulup sonra birleştiriliyor olsaydı Composite düşünülebilirdi.

Soru 3 (ekrandan, 00:50:00): Aşağıdaki ana program çalışınca ne yazdırır?

```
Karisim karaBarut = new Karisim();
karaBarut.setIsim("Kara barut");
karaBarut.maddeEkle("NaNO3", 75);
karaBarut.maddeEkle("C", 15);
karaBarut.maddeEkle("S", 10);
System.out.println(karaBarut.tarifEt());
```

> **Çözüm (derste verildi):** Çıktı `Kara barut`, `75.0gr Sodyum Nitrat`, `15.0gr Karbon`, `10.0gr Kükürt` satırlarıdır (ekrandan, 01:50:00). Gram `double` olduğu için `75.0` biçiminde yazılır; madde adı, simgeyle fabrikadan bulunan `KimyasalMadde` nesnesinden gelir.

Soru 4 (ekrandan, 01:40:00 ve 02:00:00): Kampanya 1 için zincir TrasBicak → FitnessSalonu → IndirimSporMagaza biçiminde kurulmuştur. (ERKEK, C, 23), (ERKEK, A, 53) ve (KADIN, B, 33) kullanıcılarına hangi reklamlar gösterilir?

> **Çözüm (derste verildi; python ile doğrulandı):** İlk kullanıcı 18-39 yaş arası erkek olduğu için traş bıçağı reklamını görür. İkinci kullanıcı erkek ama 53 yaşındadır; traş bıçağı ve fitness salonu (kadın arar) isteği iletir, spor mağazası indirimi gösterilir. Üçüncü kullanıcı 18-39 yaş arası kadın olduğu için traş bıçağı iletir, fitness salonu reklamı gösterilir.

Soru 5 (derste sözlü, 02:01:40): Kampanya 2'de zincirin başına "18-29 yaş arası A gelir grubu erkek" kuralıyla otomobil dergisi reklamı eklenir, ardından kampanya 1 aynen uygulanır. Aynı üç kullanıcı için sonuç değişir mi?

> **Çözüm (derste verilmedi):** Değişmez. Otomobil dergisi kuralına hiçbir kullanıcı uymaz: ilk kullanıcı C gelir grubundadır, ikincisi 53 yaşındadır, üçüncüsü kadındır. İstek kampanya 1'in zincirine iletilir ve sonuçlar yine traş bıçağı, spor mağazası indirimi ve fitness salonu olur (kurallar python ile canlandırılarak doğrulandı).

Soru 6 (ekrandan, 02:30:00): Akıllı ev örneğinde `evBoskenSokakKapisiAcildi()` ve `evdeKimseKalmadi()` senaryoları kumandaya hangi çağrıları yapar?

> **Çözüm (derste verildi):** İlki `kumanda.ac(0);` `kumanda.ac(5);` çağrılarıyla antre ışığını ve kombiyi açar. İkincisi `kumanda.hepsiniKapa();` ile listedeki bütün bileşenleri (antre, salon, mutfak, oda ışıkları, televizyon ve kombi) kapatır.

Dersin Toparlaması

Bu derste yapısal kalıplar Facade, Flyweight ve Proxy ile tamamlandı, davranışsal kalıplara Chain of Responsibility ve Command ile girildi. Facade, karmaşık bir alt sisteme tek ve üst düzey bir ön yüz sunar. Flyweight, durumu içsel ve dışsal olarak ayrılabilen çok sayıda nesneyi bir fabrika ve havuz üzerinden paylaştırarak bellek kazandırır; ancak nesne kimliğinin önemli olmadığı durumlarda kullanılabilir. Kimya fabrikası örneği, "neyin değiştiğini bul ve

onu sarmala” ilkesiyle kurulan iyi bir tasarımın Flyweight’e yalnız bir fabrika sınıfı kadar uzak olduğunu gösterdi. Proxy, asıl nesneyle aynı arayüzü sunan bir vekil koyarak erişimi denetler: resmi geç yükler, uzaktaki nesneyi temsil eder ya da izle-öde örneğindeki gibi ödeme denetimi yapar. Decorator’dan farkı, zincirin yalnız iki öğeli olmasıdır.

Chain of Responsibility, bir isteği ilişkili nesneler zincirinde onu en iyi karşılayabilecek nesneye iletir; zincirin kurulması ve izlenmesi soyut üst sınıfta yapılmalı, zincir de doğru sonlandırılmalıdır. Reklam örneği bunu her zaman uygun olan bir reklamı zincirin sonuna koyarak gösterdi. Command ise metot çağrılarını nesneye çevirir; böylece menü öğeleri ya da akıllı ev senaryoları gibi çağırıcılar, işi yapan nesnelerden ayrılır. Sıradaki davranışsal kalıp Iterator olacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.